

Student Name: M. Yousuf Lali

Student ID: m10014

Habib University
Dhanani School of Science and Engineering
Computer Architecture
EE 371 / CS 330 / CE 321 (Fall 2025)

69/100

Midterm 1

Date: 18th Oct, 2025

Total marks: 100

Obtained Marks:

Total time: 80 mins

Instructions:

1. All questions should be answered **with ink pen**.
2. Smart watches, laptops, and similar electronics are **strictly NOT allowed**.
3. Answer sheets should contain all steps, working, explanations, and assumptions.
4. Write your name and HU ID on all sheets.
5. Turn in your question paper along with your answer sheets.
6. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This exam assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	✓
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	✓
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	
CLO 4	<i>Analyze</i> the performance of cache operations	

Student Name:

Student ID:

Q1: All parts are assessing CLO 1 [Total 50 points]

Assume that the base address of a word array having 100 elements is stored in register x1. We want to assign -100 to each element of the word array using RISC-V assembly. Remember that RISC-V only supports **bge**, **bgeu**, **beq**, **bne**, **blt** and **bltu** instructions for branching.

a) Write the program using array based implementation. (10 points)

Q1
Sol: ld x1, x100
? add x5, x0, x0 // indexing.
loop
addi x6, x0, -100 // value to add
? ld x7, x100 // size of array
loop:
bge x5, x6, x5(x1) ?
addi x5, x5, 4 // increment
blt x5, x7, loop
exit:

b) Re-write the program using pointer based implementation. Assume now x1 is pointer to word array (10 points)

Q2
Sol: ld x1, x100
ld x7, x100
add x3, x1, x0
addi x6, x0, -100
loop:
sd x6 0(x3)
addi x3, x0, 4
blt x3, x7, loop
exit: ~~exit~~

What is the ending address?
7/

Student Name:

Student ID:

- c) Considering various factors of the CPU performance equation, in your opinion, which program will have better performance in terms of CPI. Restrict your answer to 5 lines (5 points)

Sol: In my opinion the pointer based approach will be relatively faster & have better performance as it has less instruction & hence a smaller CPI making the execution faster.

5/

- d) Consider this RISC-V assembly code

addi x5, x0, -100 $\rightarrow 2 \times 1$ -100

addi x6, x0, 1 $\rightarrow 2 \times 1$ 1

loop:

add x5, x5, x6 $\rightarrow 1 \times 100$ 99 98 97 96 ... 0

bne x5, x0, loop $\rightarrow 3 \times 101$

Calculate CPI for this code. Assume addi takes 2cc, add takes 1cc and bne takes 3cc. (9 points)

Sol:

Instruction Count = 4

CPU clock cycle = $2 + 2 + 100 + 303 = 407$

$$\frac{407}{4} = 101.75$$

dynamic instruction

X

5/

Student Name:

Student ID:

e) Write how each of these effect CPU performance (3 lines max, extra lines will NOT be graded)

i. Algorithm (2 points)

The performance will increase if algorithm is shorter & efficient.

ii. Compiler (2 points)

The performance will increase if, compiler is efficient & can compile large instruction set in a shorter time.

iii. Instruction Set Architecture (hardware) (2 points)

The performance will increase if the hardware can process & execute multiple instructions (multi-cycle processor).

f) Consider the following three functions, calling one another.

```
main(){
    i = i + 5;
    funct_1();
    return;
}

funct_1(){
    j = j + 6;
    funct_2();
    return;
}

funct_2(){
    k = k + 7;
    return;
}
```

Assume we want to convert the following C functions into RISC-V assembly, adhering strictly to the **RISC-V procedure calling convention**. Also, assume that the variables **i**, **j**, and **k** can be stored in **temporary registers**, and no callee-saved registers are needed for them. With these assumptions, does any of the function requires stack? If no, justify your answer. If yes, write assembly code for the function which needs stack. (10 points)

Sol: In my opinion, none of these functions require a stack as they are not returning any value & the variable are already stored in temp registers hence their value can be modified & doesn't need to be stored anywhere.

Student Name:

Q2 (40)

Student ID:

Q2: All parts are assessing CLO 2 [Total 50 points]

IEEE 754 single precision floating point format

1-bit sign	8-bit exponent	23-bits fractional part
------------	----------------	-------------------------

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} \rightarrow \text{Normal numbers}$$

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}} \rightarrow \text{Subnormal numbers}$$

- a) Write the largest sub-normal number in 32-bit binary format. Remember that for sub-normal numbers, exponent is zero. (5 points)

Sign bit: 010

exponent: 00000000

Mantissa: 111111111111111111111111

(00000000011111111111111111111111)₂

5/

- b) Convert 0x40400000 single precision floating point number to decimal. (5 points)

0x ~~000000000000000000000000~~ 40400000

8 7 6 5 4 3 2 1
40400000

Sign bit = 0

0100 0000 0100

Exponent = ^{10000000 = 128}
~~00000000~~

Mantissa = 100000000000000000000000

Sign

$$- 1^0 \times (1 + 8388608) (1 + 4194304) \times 2^{128-127} = (8388610)_{10}$$

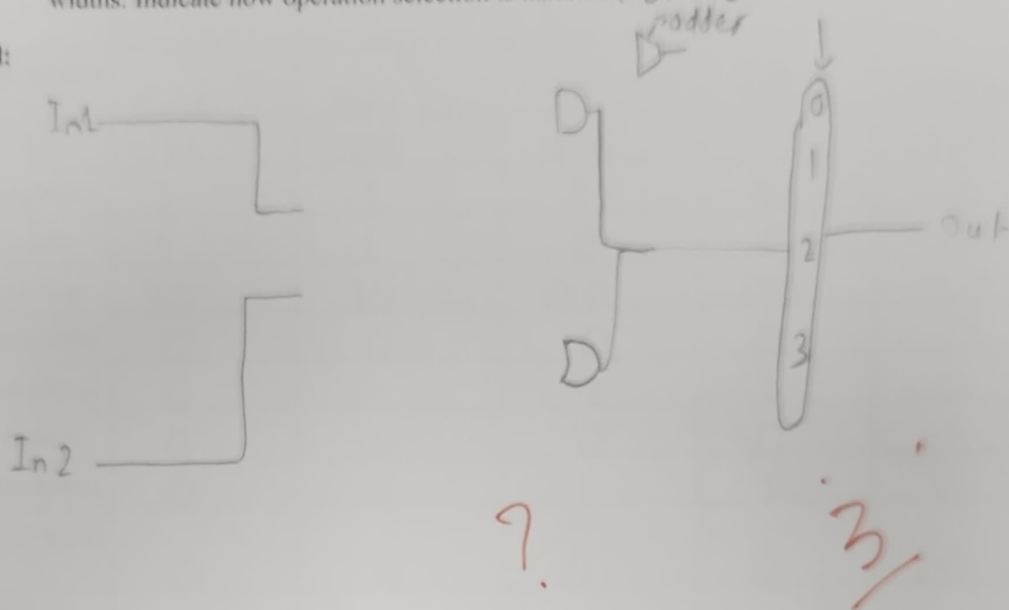
3/

Student Name:

Student ID:

- c) Design and draw a logic circuit for a 1-bit ALU that supports the following operations: AND, OR, NAND, NOR, ADD, and SUB. Use basic logic gates, multiplexers (MUXes), and a 1-bit full adder in your design. Clearly label all signals with their names and bit widths. Indicate how operation selection is handled (e.g., using control signals). (10 points)

Sol:



- d) Consider the given RISC assembly code:

```
addi x5, x0, value1      # value1 = 100, PC = 100 at cc = 0
```

```
addi x6, x0, value2      # value2 = 200
```

```
label_equal:
```

```
ld x7, 0(x5)             # value at value1 (10)
```

```
ld x8, 0(x6)             # value at value2 (20)
```

```
add x9, x7, x8
```

```
sub x10, x8, x7
```

```
and x11, x7, x8
```

```
sd x9, 0(x12)            # x12 = 300
```

```
beq x7, x7, label_equal
```

Write correct values of control and signals and current PC for the given code. Assume we have a single cycle data-path/processor. For rs1, rs2 and rd, write values in binary whereas ALU oprnd 1, 2 and result should be written in decimal (30 points)

ALUop → 10 for R type, 00 for ld/sd and 01 for branch

Student Name:

Student ID:

Sol:

	cc 0	cc 1	cc 2	cc 3	cc 4
Instruction	addi	addi	ld	ld	add
PC	100	104	108	112	116
RegWrite	1	1	1	1	1
ALUSrc	1	1	0	0	0
ALUOp	10	10	00	00	10
MemWrite	0	0	0	0	0
MemRead	0	0	1	1	0
Branch	0	0	0	0	0
MemtoReg	0	0	1	1	0
ALU oprnd 1	0	0	0	0	10
ALU oprnd 2	100	200	10	20	20
ALU result	100	200	10	20	30

	cc 5	cc 6	cc 7	cc 8	cc 9
Instruction	sub	and	sd	beq	ld
PC	120	124	128	132	108
RegWrite	1	1	0	0	1
ALUSrc	0	0	0	0	0
ALUOp	10	10	00	01	00
MemWrite	0	0	1	0	0
MemRead	0	0	0	0	1
Branch	0	0	0	1	0
MemtoReg	0	0	0	0	1
ALU oprnd 1	20	10	0	10	30
ALU oprnd 2	10	20	300	10	20 10
ALU result	10	0	300	0	10

29/